

Compiler Provenance as a Service: Decoupled Identification for Composite Provenance and Operational Resilience

Han Gao
Purdue University
West Lafayette, IN, USA
coolgao@purdue.edu

Z. Berkay Celik
Purdue University
West Lafayette, IN, USA
zcelik@purdue.com

Antonio Bianchi
Purdue University
West Lafayette, IN, USA
antonio@purdue.edu

Dave (Jing) Tian
Purdue University
West Lafayette, IN, USA
daveti@purdue.edu

Abstract

Compiler provenance aims to identify the compiler toolchain used to produce a binary, serving as a fundamental capability for binary analysis, forensics, and supply chain security. Prior studies implicitly formulate the problem as a static, one-shot classification task. Such a formulation breaks down in real-world settings, where compiler ecosystems evolve continuously, and adversaries can adaptively probe provenance logic. Furthermore, some work adopts a single-origin assumption and fails to handle binaries produced by mixed toolchains, creating blind spots in supply chain auditing.

In this paper, we study compiler provenance in real-world settings, shifting the paradigm from a static classification to a continuous service. We present S^4 , a practical provenance system built upon decoupled statistical signatures. By encapsulating each compiler as an independent identification unit, S^4 ensures incremental stability with low and controllable drift when new compilers are integrated and enables localized refinement without affecting existing units. Independent signatures provide composable evidence for composite provenance; combined with a dual-scale analysis, S^4 surfaces both primary and secondary provenance in mixed-toolchain binaries. To defend against emerging threats to provenance systems, we design a moving target defense based on randomized featurization, which limits the exposure of stable features on each query to prevent adversaries from identifying critical features by adaptive probing.

We evaluate S^4 across a wide range of compiler versions, covering all standard optimization levels. S^4 matches state-of-the-art accuracy on single-toolchain binaries and reliably handles mixed-toolchain binaries. Beyond accuracy, S^4 exhibits operational resilience through incremental stability, localized refinement, and resilience to typical adaptive probing attacks. The decoupled design inherently supports explainability, providing statistical evidence to facilitate debugging and trust in security applications.

CCS Concepts

• Security and privacy → Software reverse engineering; Software security engineering.

Keywords

Compiler Provenance, Supply Chain Auditing, Resilience

ACM Reference Format:

Han Gao, Antonio Bianchi, Z. Berkay Celik, and Dave (Jing) Tian. 2018. Compiler Provenance as a Service: Decoupled Identification for Composite Provenance and Operational Resilience. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 20 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Compiler provenance aims to identify the original compiler toolchain used to produce a binary. Identifying compiler toolchain is often the initial step in reverse engineering and directly influences downstream tasks, including decompilation [32, 52], binary code search [12, 36], and vulnerability analysis [8, 17–19, 33, 47, 67, 72, 75, 78]. For instance, de facto reverse engineering tools such as IDA [32] and Ghidra [52] allow users to specify the compiler specification before analysis to complement automatic detection. Beyond reverse engineering, compiler provenance plays a critical role in software supply chain security. Modern software ecosystems rely heavily on third-party components and precompiled libraries, which makes auditing compiler toolchains essential for detecting compromised build pipelines [74] and compiler-introduced bugs [8, 17–19, 33, 47, 67, 72, 75, 78]. As a result, compiler provenance has played a foundational role in software security.

Limitations of Existing Approaches. Existing studies typically treat provenance as a static classification task, employing Machine Learning (ML) models to pursue one-shot accuracy [12, 19, 35, 36]. However, provenance systems must operate under continuously evolving conditions in real-world settings. Compiler ecosystems regularly introduce new versions with evolving optimization and code generation logic [26, 46]. Meanwhile, adversaries can adaptively interact with provenance systems, attempting to infer the underlying decision logic for adversarial purposes (e.g., crafting evasion). In-the-wild (ITW) observations indicate that compiler provenance systems have emerged as a realistic target for attack.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

For instance, malware authors actively forge the Rich PE header [40] or deliberately strip provenance-related metadata (e.g., debug symbols or section comments) [50, 51] to impede security analysis or mislead forensics. Specifically, existing approaches have four major limitations that hinder their long-term, real-world deployment:

L1: Boundary Drift under Model Evolution. Most ML-based approaches rely on monolithic classification models. When new compilers emerge, these systems typically require global retraining or fine-tuning of the entire model [6, 13, 18, 19, 31, 35, 36, 44, 55, 57, 62, 63, 69, 77]. Such updates frequently cause unintended shifts in decision boundaries for previously supported compilers, resulting in unpredictable regressions (detailed in Section 7.3). This boundary drift poses substantial operational risks and demands costly regression testing, severely limiting long-term and real-world viability.

L2: Feature Entanglement. In monolithic models, features are globally shared across all classes. Improving performance for a specific compiler often interferes with others, as optimization or fine-tuning propagates across the entire model. This feature entanglement prevents precise, localized refinement without system-wide side effects.

L3: Single-Origin Assumption. Beyond these operational limitations, some prior work implicitly assumes that each binary originates from a single compiler toolchain [6, 12, 35, 36]. This single-origin assumption often does not hold in practice, particularly for binaries statically linked with precompiled libraries from mixed compiler toolchains. As a result, these approaches fail to capture composite artifacts, leading to blind spots in supply chain auditing.

L4: Fragility to Adversarial Probing. Existing methods implicitly have fixed decision boundaries. In particular, Pattern Matching (PM) tools such as PEiD [1], Exeinfo PE [4], and DIE [54] rely on clear indicators in binaries to identify compiler toolchain, while ML-based methods typically extract features from fixed portions of the binary. Such designs expose stable decision cues that can be adaptively probed through iterative queries [15, 16, 49, 56, 64, 66, 73]. This emerging threat has received limited attention to date, leaving existing approaches vulnerable to adaptive probing attacks.

To address these limitations, we argue that compiler provenance should be rethought as a continuously maintained service rather than a one-shot classification task. Under this paradigm, a practical provenance system must support incremental integration of new compilers with controllable regressions, enable localized refinement without system-wide side effects, support identification for mixed-toolchain binaries, and explicitly mitigate adversarial probing if it is a concern. Achieving these properties requires architectural-level rethinking rather than improvements to machine learning models.

Our Contributions. We present S^4 (Separated Statistical Signature Summary), a practical compiler provenance system designed for the Compiler Provenance as a Service (CPaaS) setting. *At its core, S^4 enforces architectural decoupling by representing each compiler as an independent statistical signature and resolving provenance via a temporal-priority policy that respects the temporal causality of compilers.* This design enables three key capabilities. First, new compiler signatures can be integrated without affecting the behavior of existing signatures, guaranteeing incremental stability. Second, each

identification unit can be independently refined without system-wide side effects. Third, independent signatures can provide composable provenance evidence, thereby handling binaries produced by mixed compiler toolchains. To address the emerging threat to provenance systems, we further design a Moving Target Defense (MTD) [71] based on characteristics of compiler provenance. By introducing query-level randomness into feature extraction, the system limits stable feature exposure and effectively mitigates adaptive probing aimed at inferring the underlying provenance logic. Our major contributions can be summarized as follows:

- We reframe compiler provenance as a CPaaS paradigm and design S^4 , a practical system that captures the operational requirements and is suitable for real-world deployment.
- We enable composite provenance in mixed-toolchain binaries by combining composable evidence with an inference-time strategy to surface both dominant and minority toolchains.
- We define the threat model for compiler provenance and introduce a moving target defense mechanism that enhances resilience against typical adaptive probing attacks.
- We conduct a comprehensive evaluation demonstrating that the decoupled identification architecture preserves accuracy, achieving performance comparable to the state of the art while exhibiting significantly stronger operational resilience.

We have released the code and data at [3].

2 Problem Statement

This section formalizes compiler provenance in the CPaaS setting. We focus on operational requirements arising from long-term, real-world deployment rather than classification accuracy alone.

Operational Requirements. We consider a compiler provenance system that operates as CPaaS and can be continuously maintained. Let $C = \{C_1, C_2, \dots\}$ denote the set of supported compilers, and let a binary B be composed of one or more components produced by compilers of C . A practical compiler provenance system must satisfy the following requirements:

R1: Incremental Stability. When a new compiler C_{new} is integrated into the system, the provenance behavior for previously supported compilers C_{old} must remain stable, avoiding systematic regressions and widespread boundary shifts. This ensures low and controllable drift, which is critical for long-term deployment without recurring regression risks.

R2: Localized Refinement. The system must support localized refinement of compiler-specific models. Refining the identification for a specific compiler C_i should not alter the decision behavior or performance of other compilers $C_j \neq C_i$. This requirement reflects the need for precise, low-risk maintenance in operational environments, where targeted refinement is often required to correct emerging failures or harden the system against adversarial behaviors without introducing system-wide side effects.

R3: Composite Provenance. The system must explicitly support composite provenance for mixed-toolchain binaries. In practice, binaries often incorporate static libraries (e.g., system libraries) compiled with different compiler toolchains. A provenance system should therefore report evidence for mixed-toolchain binaries, rather than forcing a single-origin assumption.

R4: Resilience to Adaptive Probing. The system should account for adversarial interaction. Adversaries may iteratively query the system and attempt to infer its decision logic. A robust provenance system should demonstrate resilience against adaptive probing by enforcing a high probing cost and a low probing success rate.

Collectively, these requirements not only enforce correctness and resilience under continuous deployment, but also induce explainability as a first-class system property. We treat explainability as a derived operational property and evaluate in Section 7.7.

3 System Design and Overview

This section provides a high-level overview of S^4 and the design philosophy underlying its architecture. We aim to make the system semantics explicit rather than describing technical details.

3.1 Design Rationale

The operational requirements in Section 2 expose fundamental limitations of monolithic approaches: requirements such as incremental stability (R1), localized refinement (R2), and composite provenance (R3) cannot be naturally satisfied by globally coupled decision logic. *These limitations arise from architectural coupling rather than from the choice of features or classifiers.* To address this gap, S^4 adopts a decoupled identification architecture in which provenance evidence for each compiler is modeled independently. By preserving independence throughout training, inference, and maintenance, the system supports incremental evolution and localized maintenance without perturbing existing behavior. This architectural choice also allows the system to naturally identify composite provenance when multiple compiler toolchains contribute to a single binary.

In addition to R1, R2, and R3, CPaaS may operate under adversarial settings in which the provenance system is exposed to adaptive probing (R4). To enhance resilience against adaptive probing attacks, S^4 optionally incorporates an inference-time hardening mechanism that limits stable feature exposure across queries. This mechanism is orthogonal to the decoupled identification architecture and is not mandatory for correctness.

3.2 System Pipeline

Figure 1 illustrates the pipeline of S^4 . We refer to each compiler-specific evidence unit as a *signature*. At a high level, the system processes an input binary (fully stripped of symbols) through two consecutive stages: (i) per-compiler signature evaluation, and (ii) resolution over the set of positive signature results.

Per-Compiler Signature Evaluation. Given a stripped binary as input, S^4 evaluates it against a bank of compiler signatures. Each signature corresponds to a specific compiler toolchain and independently examines the binary. Each signature produces a binary outcome indicating whether the binary exhibits evidence consistent with that compiler.

Collection of Positive Results. After all signatures have been evaluated, S^4 collects the set of compilers whose signatures return positive decisions. This set represents the observable provenance evidence exposed by the system for the given binary. At this stage, the system does not attempt to resolve or prioritize the results; it simply records which signatures have fired.

Resolution and Output. The collected positive results are then processed by a resolution module, which determines the final output presented to the user. Depending on the configured output semantics, the system may return a single compiler label, a ranked list of multiple compilers, or abstain when no signature produces a positive result.

Overall, the pipeline separates evidence generation from decision resolution. The motivations, semantics, and operational implications of this design are discussed in the following sections.

4 Decoupled Identification Architecture

Most prior compiler provenance systems resolve provenance through a monolithic classification model learned jointly over all supported compilers. Such coupling ties provenance to a shared decision model and often enforces mutual exclusivity, so adding a new compiler or improving one target compiler typically requires global retraining or fine-tuning. In contrast, S^4 adopts a different abstraction, explicitly modeling compiler toolchain evolution. This section presents the core design underlying S^4 : *decoupled identification*. Instead of embedding all provenance logic into a monolithic classifier, S^4 represents each compiler with an independent evidence unit and aggregates evidence with a temporal-priority resolution policy. This design decouples maintenance from global retraining, enabling continuous evolution in CPaaS settings.

4.1 Decoupled Evaluation

As illustrated in Figure 1, S^4 maintains a bank of compiler signatures, each corresponding to a supported compiler toolchain. Under decoupled evaluation, provenance is not resolved through a single global classifier. Instead, the association between the input binary and each compiler signature is evaluated independently by posing the following question: *Does the input binary contain evidence consistent with this compiler?*

Formally, let $C = \{C_1, \dots, C_n\}$ denote the set of supported compilers, ordered by release time, with signatures integrated following the same temporal order. Each compiler C_i is associated with a signature Sig_i , which serves as an independent evidence unit. Given an input binary B , the signature examines the binary and produces a binary decision indicating whether sufficient evidence is observed. Formally, this decision is defined as

$$\text{Sig}_i(B) \in \{+, -\}.$$

A signature encapsulates all logic required for this decision, including feature extraction $f_i(\cdot)$, and decision model $g_i(\cdot)$. Different signatures may adopt different feature modeling strategies and classifiers, thereby preserving heterogeneity across compilers. To enforce architectural decoupling, signatures expose only their binary outcomes and do not reveal any internal states. S^4 evaluates all signatures on the input binary and collects the set of positives

$$\mathcal{P}(B) = \{C_i \mid \text{Sig}_i(B) = +\}.$$

Since evidence is not forced into a mutually exclusive label, $\mathcal{P}(B)$ may contain multiple compilers or be empty. The positive set $\mathcal{P}(B)$ is then processed by a temporal-priority resolution policy.

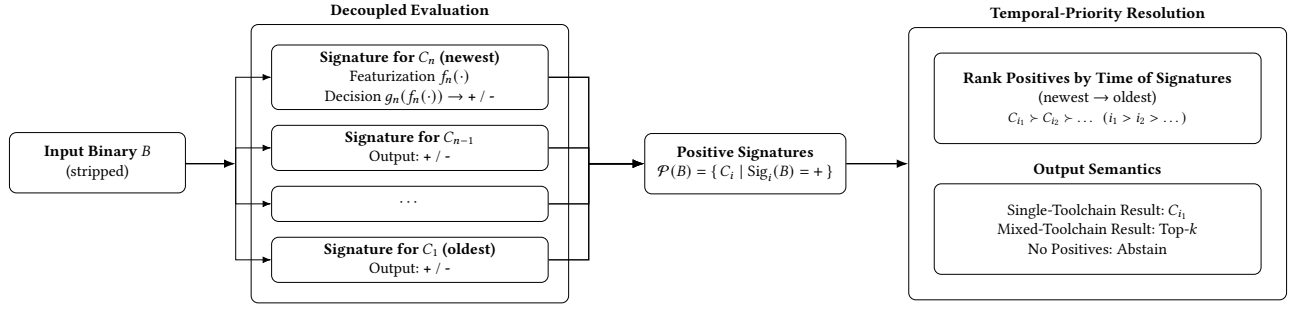


Figure 1: System pipeline of S^4 . The input binary is evaluated independently by per-compiler signatures. Positive results are collected and passed to a resolution module, which produces the final provenance output.

4.2 Temporal-Priority Resolution

Since signatures are integrated over time, they exhibit temporal asymmetry: newer signatures can be derived using earlier compilers as negatives, but older ones have no access to data from future compilers. Consequently, older signatures are more prone to spurious positives on binaries produced by newer toolchains, because they are not trained against future compilers.

S^4 reduces this failure mode using a temporal-priority policy that prioritizes newer signatures. Specifically, temporal-priority resolution ranks the positive compiler signatures from newest to oldest, yielding an ordered list $C_{i_1} \succ C_{i_2} \succ \dots$ with $i_1 > i_2 > \dots$. This ordered list supports multiple output semantics. When a single-toolchain decision is required, S^4 outputs the newest positive compiler C_{i_1} . When mixed-toolchain output is enabled, S^4 returns the top- k compilers in the ordered list, where the first compiler is treated as the primary provenance and the remaining compilers provide prioritized secondary evidence. If the positive set $\mathcal{P}(B)$ is empty, the system produces no attribution. This situation arises when no signature yields sufficient evidence (e.g., the input binary is produced by an unsupported compiler toolchain). In such cases, S^4 explicitly abstains and treats abstention as a first-class outcome to avoid overconfident attributions.

In summary, temporal priority is implemented purely at the aggregation stage and aligns with the temporal asymmetry in signature building. It allows new signatures to be integrated without modifying existing ones, thereby preserving incremental stability under continuous integration.

4.3 Machine Learning as Statistical Signatures

Prior work has shown that learning-based techniques are highly effective in identifying compiler provenance by capturing compiler-induced statistical patterns in binary code [12, 18, 19, 31]. Rather than treating machine learning models merely as classifiers, we adopt a different abstraction and view each model as a *statistical signature* that encapsulates the characteristic code patterns of a specific compiler configuration. Under this abstraction, a signature is a self-contained decision unit that is trained, stored, and evaluated independently. Its role is not to compete with other signatures, but to answer a single binary question: whether an input binary matches the compiler represented by the signature. Consequently, the overall identification process does not rely on a global normalization

or competition mechanism such as softmax, nor does it require signatures to share feature representations or learning algorithms. This decoupled design allows signatures to evolve independently and enables flexible integration of heterogeneous mechanisms.

In principle, different signatures may adopt different feature modeling and classification models. However, in our experimental evaluation, we intentionally instantiate all signatures using the same feature representation and learning algorithm, following the state-of-the-art approach described in Section 7.1. This choice is made to control experimental variables rather than to constrain the design space. Importantly, this controlled instantiation does not violate the decoupled nature of signatures, as each signature remains an opaque decision unit whose internal state is not observable outside its binary output. By fixing the learning architecture across signatures, we eliminate confounding effects introduced by feature engineering and model selection. This allows us to isolate and rigorously evaluate the impact of the proposed decoupled identification architecture and temporal-priority resolution policy.

4.4 Operational Guarantees

The decoupled identification architecture directly yields several operational guarantees required by CPaaS.

Incremental Stability. Each statistical signature is trained and deployed independently. Integrating a new compiler signature does not modify or retrain existing signatures. Consequently, integrating a new compiler affects only the new signature and the aggregation layer, leaving existing identification units stable.

Localized Refinement. Because each signature has its own parameters and is trained and deployed independently, refining a specific compiler signature affects only that signature. Improvements in accuracy or robustness do not propagate to unrelated compilers, enabling precise and low-risk maintenance.

Composite Provenance. Because evidence is not forced into a single label, S^4 can return multiple positive compilers as a temporally ranked evidence list. This directly supports mixed-toolchain provenance while preserving the same maintenance model: integrating or refining one compiler does not require retraining a global classifier.

Together, these properties demonstrate that decoupled identification is a foundational architectural choice rather than an auxiliary design feature. By enforcing independence throughout the system

life-cycle, S^4 delivers compiler provenance as a CPaaS under real-world conditions.

4.5 Distinction from One-vs-Rest Classification

One-vs-rest (OvR) is a widely used classification framework for extending binary classification to multiclass settings [30], where a separate classifier is trained to distinguish each class from all others [65]. While the decoupled identification architecture shares this decomposition, it differs from OvR in three aspects: *training procedure*, *resolution mechanism*, and *heterogeneity constraints*.

Training Procedure. OvR assumes a symmetric training setting, where each binary classifier is trained against negatives drawn from all other classes [30, 65]. As a result, although the overall classification is decomposed into per-class classifiers, the training procedure remains globally coupled: each binary classifier relies on the full set of other classes so that it is supposed to be retrained whenever the class set evolves. In contrast, S^4 adopts an asymmetric and fully decoupled training procedure: when training the signature for the compiler C_i , we use only binaries from compilers released prior to C_i as negatives. This design allows new compilers to be incorporated without retraining existing signatures, which lays the foundation for operational resilience.

Resolution Mechanism. In the conventional OvR framework, final predictions are determined either by taking the argmax of the classification scores or by applying a decision threshold to each score (for multilabel tasks) [30, 65]. However, these methods are not applicable because of the asymmetric training procedure. As discussed in Section 4.2, temporal asymmetry in signature integration allows newer signatures to use binaries from earlier compilers as negatives, whereas older signatures, at the time of their training, cannot access binaries produced by later compilers. Consequently, scores from different signatures do not represent calibrated evidence on a common scale. To address this, S^4 adopts a temporal-priority resolution, ranking positive signatures from newest to oldest and prioritizes newer compilers. This resolution policy aligns with the temporal asymmetry in continuous integration and enables prioritized secondary evidence for mixed-toolchain binaries.

Heterogeneity Constraints. While OvR permits heterogeneous models in principle [30], mainstream implementations such as scikit-learn [65] typically enforce a shared feature representation and learning objective to ensure score comparability, thereby limiting model heterogeneity in practice. In contrast, S^4 treats each statistical signature as an independent decision unit, allowing it to adopt its own feature representation and classification model. This design enables heterogeneous instantiations tailored to the difficulty and distinctiveness of individual compilers.

In summary, decoupled identification can be viewed as a task-specific variant of OvR rather than a straightforward application. While both employ binary classifiers, they differ in training procedure, resolution mechanism, and heterogeneity constraints, leading to improved adaptability in compiler provenance.

5 Mixed-Toolchain Identification

Mixed-toolchain binaries typically exhibit an imbalance: The primary compiler dominates the code footprint and determines the

major characteristics of the binary, while secondary compiler signals introduced by static linking could be weak from a global view. Accordingly, S^4 adopts a dual-scale inference-time analysis that identifies primary and secondary provenance in stages.

Primary Compiler. The primary (assuming host) compiler is identified using the standard single-toolchain pipeline. Given an input binary, S^4 globally analyzes it and produces a positive decision. This practice captures pervasive stylistic patterns induced by the major compiler and is shown to be robust to partial interference from secondary components (detailed in Section 7.5).

Secondary Compiler. The signal of the secondary (assuming library) compiler is often weak and may be diluted under global analysis. Nevertheless, prior studies indicate that static linking exhibits strong link-time locality: functions originating from the same object file or library are typically placed in contiguous regions of the binary [38, 60]. Code generated by the secondary compiler tends to be concentrated in localized regions rather than uniformly interleaved. This structural property enables the recovery of secondary compiler provenance through localized analysis, where region-level inspection can amplify weak signals that are otherwise obscured in a global view. Specifically, S^4 performs a window-based analysis parameterized by a window size w and a sliding stride s . Given an input binary, we partition it into a sequence of windows, each covering a fixed fraction w of the binary, with adjacent windows offset by s . Windows may overlap when $s < w$, enabling finer-grained coverage of localized regions. Each window is analyzed independently using the same decoupled identification architecture. In our experiment, we set the sliding window size w to 20% of the binary size and the stride s to 10%, balancing locality for capturing statically linked library code with stable coverage. For each window, if the top-ranked signature corresponds to the primary compiler (which is already identified by global analysis), the window is treated as host-dominant and excluded from further consideration. The remaining window-level predictions are then aggregated via majority voting to determine the secondary compiler. This aggregation reflects the intuition that secondary compiler signals, while weak at the global scale, manifest consistently across contiguous windows corresponding to statically linked library code.

6 Hardening Against Adaptive Probing

This section describes a hardening mechanism against adaptive probing in adversarial settings. The goal is not to fully prevent attacks, but to increase the cost and uncertainty of inferring provenance logic. All core properties under benign settings (decoupled identification and temporal-priority resolution) remain unchanged.

6.1 Threat Model

We consider an adversary who can interact with the provenance system through adaptive queries to probe its decision boundaries. **Adversary Goal.** Given a target binary, the adversary aims to identify decisive regions that determine the result. We term this process *critical feature localization*, which serves as a common precursor to crafting evasive variants.

Adversary Capabilities. The adversary has black-box access to the provenance system and observes only the final output compiler

label(s) returned by the system (including abstention when applicable). The adversary cannot observe any internal states, including intermediate evidence, or confidence scores.

Attack Strategy. The adversary adaptively issues queries and exploits consistency of outputs across queries to refine hypotheses about which regions are decisive. Such strategies exploit the fact that conventional provenance systems expose stable feature evidence: repeated queries on the same binary yield identical feature representations and thus predictable decision behavior.

Probing Granularity. We assume that the adversary performs probing at the instruction level, as instruction is the semantically meaningful primitive of binaries. If an adversary localizes a set of decisive instructions, these instructions can be directly subjected to semantics-preserving rewriting, such as instruction substitution or reordering, to enable follow-up evasion attempts. This probing-then-rewriting workflow aligns with real-world evasion practices [49, 64, 66, 73], where probing is implemented by masking instruction regions to locate influential content. Our evaluation focuses on the *feature space induced by the provenance pipeline*. Recall that the goal of adversaries is probing, localizing influential regions rather than achieving full evasion. We do not model how a set of instructions is concretely rewritten into a functional evasive binary.

Attack Constraints. The adversary is constrained by (i) a finite query budget Q , which bounds the number of queries, and (ii) a perturbation budget P , which represents an abstract bound on feature-level manipulation. This abstraction captures probing progress without committing to concrete binary rewriting realization. Under this model, we empirically evaluate whether hardening increases probing cost and degrades probing effectiveness.

6.2 Moving Target Defense

To counter adaptive probing attacks that exploit stable inference behavior, S^4 can optionally incorporate a moving target defense (MTD) mechanism that deliberately introduces controlled randomness at inference time. Adaptive probing is effective when the system exposes a stable mapping from an input to a fixed featurization, leading to a fixed decision procedure. In that case, an adversary can correlate small input changes with stable output changes and gradually localize critical regions. To disrupt this convergence, S^4 randomizes provenance evidence exposure on each query. Our defense is designed specifically to impede critical feature localization. By limiting stable feature exposure across queries, the system disrupts the adversary's ability to reliably correlate query inputs with consistent decision signals, turning probing into noisy inference. This design is motivated by an empirical observation in compiler provenance: compiler-induced patterns tend to be pervasively distributed across the binary, such that analyzing only a partial code footprint is often sufficient for reliable attribution [18, 19, 31, 44, 48, 55, 62, 69]. This observation enables a partial, query-dependent exposure of evidence, preserving the integrity of provenance signals.

Implementation. The MTD mechanism is implemented via *randomized featurization* at the function granularity. Given an input binary B , S^4 first identifies its constituent functions through disassembly. Instead of deterministically extracting features from a fixed subset of functions (typically most or all functions in the binary),

the system uniformly samples functions at random and progressively aggregates their instructions until the cumulative instruction count reaches a predefined threshold T . Feature extraction (e.g., n -gram statistics) is then performed solely on this sampled instruction pool. Formally, let $\mathcal{F}(B)$ denote the set of discovered functions in B , and let π be a random permutation of $\mathcal{F}(B)$ drawn independently for each query. The sampled instruction set is constructed by traversing the functions in the order defined by π and accumulating their instructions until the total number reaches or exceeds T . The resulting feature vector is therefore query-dependent, even for the same binary B . This design preserves compiler-specific patterns in expectation, while preventing any single query from revealing the full set of potentially decisive evidence.

Effect. Randomized featurization reduces stable evidence exposure across queries, forcing a probing adversary to issue more queries to achieve comparable coverage of decisive signals while still facing uncertainty from random overlap in sampled instruction sets. By operating at function granularity, the mechanism preserves compiler-specific code patterns and retains sufficient evidence for accurate provenance (via threshold T). As quantified in Section 7.6, this controlled randomness significantly increases probing cost and degrades probing effectiveness, thereby providing system-level resilience against adaptive probing in adversarial scenarios.

Trade-off. Randomized featurization may affect accuracy when T is too small. In practice, T controls the trade-off: larger T improves fidelity by exposing more evidence per query, while smaller T increases uncertainty for probing adversaries. We treat T as a hyperparameter that can be tuned based on the validation set.

Compatibility. The MTD mechanism is orthogonal to the decoupled identification architecture. It changes only the featurization of each signature (i.e., $f_n(\cdot)$). Signatures remain independent evidence units, and temporal-priority resolution remains unchanged.

Position. This hardening is designed to impede *adaptive probing for feature localization*. It does not guarantee security against all forms of evasion, especially if an adversary has strong side information, unrestricted query budgets, or capabilities beyond black-box interaction. Nevertheless, as an *optional* configuration for CPaaS deployments where adversarial probing is a concern, randomized featurization provides a model-agnostic way to harden against adaptive probing attacks.

7 Evaluation

We evaluate S^4 based on the following research questions, each corresponding to a core capability required by CPaaS.

RQ1: Decoupled Fidelity. Does decoupled identification faithfully preserve compiler provenance capability compared to state-of-the-art approaches?

RQ2: Incremental Stability. Do existing approaches exhibit decision drift when new compiler toolchains are added? Does S^4 provide incremental stability when scaling to new toolchains?

RQ3: Localized Refinement. Can S^4 support controllable localized refinement of compiler signatures without system-wide side effects?

RQ4: Composite Provenance. How effectively does S^4 handle composite provenance for binaries compiled with mixed toolchains?

RQ5: Probing Resilience. Can moving target defense mechanism effectively mitigate adaptive probing?

RQ6: Explainable Decision. Can S^4 provide explainable evidence for debugging and understanding?

7.1 Experimental Setup

Environment. We conduct experiments on a Ubuntu 22.04 server equipped with AMD EPYC 7773X CPUs and 1024GB memory. Binary analysis is performed using IDA Pro (v8.3) and Ghidra (v11.0.3). **Dataset.** The experiments are based on the BinKit dataset [43], the largest binary dataset widely used in evaluating binary-level tasks and focus on x86-64 binaries¹. There are 26,676 binaries compiled from 50 projects under various compiler settings, including 18 compilers (8 GCC + 10 Clang) across all standard optimization levels (O0–3, Os, Ofast). Each compiler class contains 1,482 binaries, with a 70%-15%-15% split for training, validation, and testing. All binaries are fully stripped of symbols, including superficial indicators such as compiler names, to avoid any risk of information leakage.

Feature Modeling and Classifier. Our contributions lie at the paradigm and architectural level, rather than specific feature modeling or employing more advanced machine learning models². We align with the state of the art, PassTell [19], using n -gram statistics to extract features from instructions and a Gradient Boosted Decision Tree (GBDT) classifier for classification. This alignment ensures that any observed differences stem from the proposed architecture and policy, rather than feature engineering or classifiers.

Baselines. We include prior work in our baselines as long as their code is publicly available. Specifically, we consider six ML-based methods (PassTell [18, 19], ORIGIN [62], O-glassesX [55], DComP [12], DComP++ [44], and BinAlign [35]) and two PM-based methods (YarGen [21] and AutoYara [58]). All baselines are retrained/rebuilt on the same dataset as S^4 to ensure a fair comparison.

Machine Learning Baselines. PassTell is the state-of-the-art approach in compiler provenance and serves as our primary baseline. PassTell employs n -gram statistics to extract features primarily from instructions and utilizes a GBDT classifier to identify compiler provenance. In addition to PassTell, we further include five ML-based methods whose code is publicly available for single-toolchain comparison. ORIGIN [62] employs a probabilistic graphical model to model the relationship of adjacent functions and performs binary-level classification. O-glassesX [55] employs deep neural networks to analyze instruction sequences, considering both opcodes and operands. Its official repository provides two versions for input lengths of 16 and 64 [79], both included in our baselines. DComP [12] uses the combination of mnemonics, registers, and function lengths as features and employs lightweight neural networks for classification. DComP++ [44] is based on DComP [12], additionally introducing statistical weighting to score opcodes and constructing a hierarchy of linear SVMs for classification. BinAlign [35] proposes streamlined alignment padding features and achieves promising results. The original implementations of O-glassesX, DComP++, BinAlign, and PassTell do not align with our experiments in terms of classification granularity or binary format. We extended their code to support binary-level compiler identification on x86-64 ELF files.

Pattern Matching Baselines. Pattern matching also plays an indispensable role in compiler provenance. We consider two state-of-the-art approaches, YarGen [21] and AutoYara [58], which can automatically derive pattern matching rules from given binaries for identification. YarGen derives rules by identifying the fragments found in positives and removing known fragments that also show in negatives. AutoYara leverages large n -grams and a bi-clustering algorithm to create rules for pattern matching. Nevertheless, their features remain at the syntactic level (i.e., relying on exact matches of specific byte sequences), in contrast to statistical patterns.

7.2 RQ1: Decoupled Fidelity

Methodology. To evaluate whether decoupled identification preserves accuracy, we compare S^4 with ML- and PM-based baselines on standard single-toolchain binaries. All systems are trained and evaluated on identical data splits. We evaluate with standard classification metrics: Precision (P), Recall (R), F1 score (F), and Accuracy (Acc). While we report these metrics, our goal here is not to optimize the performance, but to evaluate fidelity, i.e., whether decoupled identification preserves the ability to capture compiler-induced evidence without relying on globally coupled decision logic.

Results. Table 1 reports the result. Overall, S^4 achieves performance that is on par with the state-of-the-art approach. Compared to PassTell, S^4 exhibits nearly identical performance, with differences below 0.3% on all metrics. This result shows that decoupling compiler identification into independent statistical signatures preserves identification fidelity, despite removing global competition and shared decision boundaries that are commonly assumed to be necessary for high accuracy.

Compared to prior ML-based methods, S^4 consistently improves both precision and recall, indicating that independent evidence modeling does not introduce bias toward either conservative or aggressive attribution. In contrast, PM-based methods exhibit highly imbalanced behavior. YarGen achieves high precision but low recall (high false negatives), while AutoYara exhibits the opposite trend (high false positives). These results confirm that pattern matching is fundamentally limited: once binaries are stripped and shallow indicators removed, combining syntactic-level features is unlikely to yield effective rules. Notably, S^4 -MTD, which integrates the MTD mechanism described in Section 6.2, incurs only a marginal reduction in accuracy compared to S^4 . The performance gap between S^4 and S^4 -MTD is within 1.7% across all metrics, demonstrating that the MTD mechanism introduces minimal overhead on accuracy. Overall, decoupled identification preserves accuracy compared to state-of-the-art. Importantly, this fidelity is achieved without relying on globally coupled decision logic, providing the foundation for the operational guarantees required by CPaaS.

Time Efficiency for Integrating New Compilers. We evaluate the time overhead incurred when extending the system to support an additional compiler. Benefiting from the decoupled identification architecture, S^4 requires training only a new binary classifier for each newly introduced compiler, leading to low integration cost. In contrast, monolithic models have to be fully retrained to incorporate the new class, resulting in substantial overhead. We use PassTell as the primary baseline, as it employs the same feature extraction

¹Note that S^4 is not architecture-specific and can be adapted to other platforms.

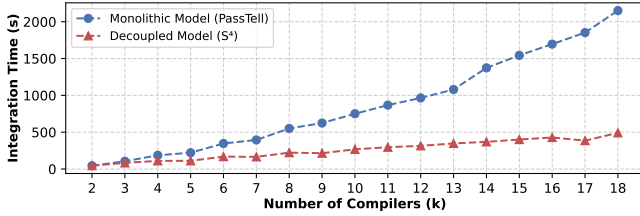
²We emphasize that our contributions are feature- and model-agnostic.

Table 1: Evaluation on Compiler Identification

Category	Method	P (%)	R (%)	F (%)	Acc (%)
PM based	YarGen [21]	69.64	28.48	20.52	79.58
	AutoYara [58]	6.97	70.35	12.56	43.58
ML based	ORIGIN [62]	63.27	60.88	61.03	60.86
	O-glassesX (16) [55]	43.67	15.44	11.61	15.00
	O-glassesX (64) [55]	41.31	29.52	25.06	30.36
	DICoMP [12]	72.97	71.32	71.44	71.32
	DICoMP++ [44]	60.21	60.48	59.70	60.38
	BinAlign [35]	55.30	51.40	52.75	51.68
	PassTell [18, 19]	89.09	88.96	88.90	88.96
	S^4	89.07	88.66	88.65	88.66
	S^4 -MTD	87.42	87.46	87.31	87.46

and classification models as S^4 . Their comparable accuracy also provides a sound basis for comparing time efficiency.

Figure 2 shows the integration time as the number of compilers increases. Under aligned settings, retraining the monolithic model takes $1.28\times$ (at $k = 3$) to $4.42\times$ (at $k = 18$) longer than training a single new classifier. This gap widens as the number of classes grows, demonstrating the time efficiency of the decoupled design.

**Figure 2: Integration time of monolithic model and S^4 .**

Regarding Optimization-Level Identification. Existing compiler provenance studies often consider compiler identification jointly with optimization-level identification. To ensure a comprehensive comparison, we additionally evaluate S^4 on optimization-level identification under the same experimental setting. This experiment is not the primary focus of our work. Nevertheless, experiments show that S^4 achieves accuracy comparable to state-of-the-art in optimization-level identification, indicating that the decoupled identification architecture generally does not compromise compiler provenance. Detailed results are reported in Appendix A.

Answer to RQ1: S^4 preserves compiler provenance fidelity under decoupled identification, achieving accuracy comparable to state-of-the-art while largely reducing time overhead when supporting newly introduced compilers.

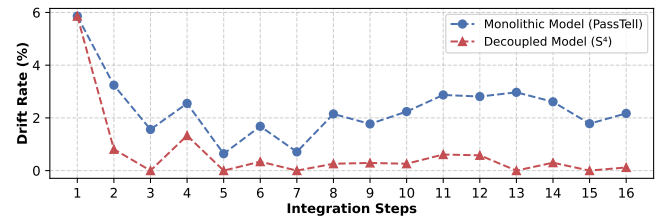
7.3 RQ2: Incremental Stability

Methodology. This experiment examines whether extending the system to support the identification of new compiler toolchains introduces unintended behavior changes for previously supported ones. We simulate a realistic CPaaS deployment scenario in which compilers are incrementally integrated over time. The evaluation consists of two complementary parts. First, we assess the existence

and severity of retraining-induced drift in monolithic methods. Starting from an initial set of compilers (i.e., the earliest two compilers in our experiment), we incrementally introduce new compiler toolchains. At each integration step, monolithic baselines are retrained to support the identification of the newly added compiler following their standard training procedures. After retraining, we evaluate the updated model on the original test set of previously supported compilers. Second, we evaluate S^4 under the same incremental integration settings. For S^4 , integrating a new compiler only introduces a new statistical signature and does not retrain or modify existing signatures. After each integration step, we re-evaluate the same original test set to examine whether predictions for previously supported compilers change.

Metrics. We quantify retraining-induced regression using the drift rate, defined as the fraction of test samples that were correctly identified before integrating a new compiler but become misclassified afterward. We emphasize that drift rate is intentionally asymmetric and is not an accuracy metric. Standard metrics such as precision, recall, and F1 measure the absolute correctness of a model at a single point in time, but they cannot capture behavioral instability across system updates. In CPaaS deployments, even an update that improves overall accuracy may be operationally unacceptable if it silently flips previously correct predictions on legacy workloads, since downstream pipelines (e.g., disassembly, decompilation, vulnerability scanning) may have been validated against those predictions. Drift rate directly captures this regression risk and is conceptually aligned with regression testing in software engineering, rather than with classification accuracy. We report standard P/R/F1 separately in RQ1 to characterize accuracy; here we deliberately isolate the stability dimension.

Baselines. For the drift analysis, we use PassTell as the representative of monolithic models, as it is the state-of-the-art approach and performs best in RQ1. We exclude other baselines from this analysis because their initial performance is significantly lower than that of S^4 and PassTell, which would render drift comparisons less informative. Evaluating drift on the comparable methods ensures that any observed regressions stem from the inherent limitations of globally coupled retraining during incremental integration, rather than from insufficient model capacity.

**Figure 3: Drift analysis of monolithic model and S^4 .**

Results. Figure 3 shows the drift behavior as new compilers are incrementally added. We observe a clear divergence in drift behavior between monolithic and decoupled identification as the system evolves. At the initial integration step ($t = 1$), both monolithic model and S^4 exhibit a relatively high drift rate. This phenomenon reflects a cold-start phase. The first newly introduced compiler (clang-5.0)

is highly similar to an existing one (clang-4.0), and the negative coverage for the new class or signature is minimal. As a result, both approaches experience pronounced boundary interference: PassTell undergoes substantial global boundary reshaping due to retraining, while S^4 incurs resolution-induced overrides caused by false positives of the newly added signature on a small fraction of existing samples. As more compilers are incrementally integrated, the drift trends of the two approaches diverge significantly. For the monolithic model, step-wise drift remains consistently high throughout the integration process, with an average drift rate of approximately 2%. This behavior is intrinsic to monolithic retraining: each integration step requires retraining the global classification model, which continuously rebuilds decision boundaries for all previously supported compilers, leading to persistent regression risk. In contrast, S^4 exhibits a rapidly decreasing and stabilizing drift trend. After the cold-start phase, decision drift drops sharply and converges to a low and stable level of around 0.3%. Moreover, S^4 achieves zero drift at multiple integration steps ($t = 3, 5, 7, 13, 15$), indicating that no regressions are introduced for previously supported compilers at these stages. This stability arises from decoupled identification, which preserves existing statistical signatures and isolates system evolution to newly added components. In S^4 , drift is not caused by changes to existing classifiers, but instead stems solely from occasional false positives of newly integrated signatures on old data, which are surfaced through the temporal-priority resolution policy (as detailed in Section 4.2). Such drift is localized, controllable, and interpretable.

Overall, these results demonstrate that decoupled identification fundamentally alters the drift dynamics of compiler provenance systems. While both approaches are susceptible to cold-start effects, S^4 rapidly stabilizes as the system grows, whereas monolithic models continuously introduce regressions due to global retraining.

Answer to RQ2: Monolithic methods suffer from retraining-induced decision drift. S^4 significantly reduces decision drift under incremental compiler integration, and frequently achieves zero drift at individual integration steps, thereby satisfying the incremental stability requirement defined in Section 2.

7.4 RQ3: Localized Refinement

Problem. To evaluate whether S^4 supports localized refinement without inducing system-wide side effects, we focus on a concrete regression observed in RQ2. As discussed in previous section, the first integration step exhibits the highest drift rate. Specifically, at integration step $t = 0$, the system supports two compilers, gcc-4.9.4 and clang-4.0, and achieves 100% accuracy on their test set. When a new compiler, clang-5.0, is introduced at step $t = 1$, the overall accuracy on previously supported compilers drops to 92.94%, corresponding to a drift rate of 5.86%. Manual inspection reveals that this degradation is primarily caused by false positives from the newly introduced clang-5.0 signature on clang-4.0 binaries. Due to temporal-priority resolution, these false positives override correct attributions from older signatures, leading to decision drift. This scenario reflects a realistic CPaaS maintenance setting, where integrating a new compiler introduces regressions on legacy workloads. **Methodology.** To mitigate the observed regression, we perform a localized refinement that targets only the clang-5.0 signature,

while keeping all other signatures unchanged. We first diagnose the failure mode by applying the clang-5.0 signature to clang-4.0 binaries from the training set. Among these samples, we identify those that receive high confidence scores (e.g., greater than 0.2 in our experiment) from the clang-5.0 signature. These samples represent hard negatives that are most likely to trigger false positives after integration. To refine the clang-5.0 signature, we retrain it by increasing the weights of these hard-negative samples. The refinement is lightweight and conservative, introducing no new data or model changes, and does not affect any other compiler signatures. No validation or test data is used during refinement.

For comparison, we apply the same hard-negative patch data to the monolithic model (PassTell). Since PassTell employs a globally coupled decision model, refinement necessarily involves retraining the entire model based on the weighted training set. We emphasize that the specific refinement method itself is not the focus of this experiment. Rather than maximizing performance gains for a single compiler, our goal is to demonstrate that S^4 enables precise localized refinement without interference to other signatures.

Results. After localized refinement, S^4 exhibits a consistent reduction in decision drift. The drift rate decreases from 5.86% to 4.05%, and the overall accuracy on previously supported compilers increases from 92.94% to 93.09%. Importantly, this improvement is achieved without introducing extra regressions on unrelated compilers. We measure the prediction change rate on all test samples not involving clang-5.0 and observe that it remains negligible, indicating that the effects of refinement are strictly confined to the targeted signature. This confirms that localized refinement in S^4 does not perturb the behavior of unrelated compiler signatures. In contrast, retraining the monolithic model reduces the targeted regression but induces unintended prediction changes across previously supported compilers. This behavior is intrinsic to globally coupled models, where modifying the decision boundary for one class inevitably affects others.

Answer to RQ3: S^4 enables controllable localized refinement: regressions introduced by a newly integrated compiler can be mitigated by refining only the responsible signature, without inducing system-wide changes, in contrast to monolithic methods.

7.5 RQ4: Composite Provenance

Methodology. This experiment evaluates the performance of S^4 when a binary contains code produced by multiple compiler toolchains. We focus on the common two-toolchain setting: a binary is compiled with a *host* compiler, while it statically links some libraries (e.g., system libraries) produced by a different *library* compiler. Composite provenance introduces two different challenges. First, the presence of library code may interfere with identifying the primary compiler, potentially degrading provenance reliability. Second, the signal of the secondary compiler is often weak, raising the question of whether such signals can be surfaced without retraining or modifying compiler signatures. Accordingly, we decompose this RQ into two complementary questions: (i) Can S^4 reliably identify the primary compiler when a binary contains code produced by another toolchain? (ii) Can S^4 surface meaningful secondary compiler provenance beyond the dominant signal?

To systematically evaluate S^4 in composite provenance, we conduct two complementary experiments. First, we construct a synthetic dataset with controlled library-code proportions to measure their effects on composite provenance. Second, we conduct a case study on executable binaries from real-world projects built with mixed toolchains, providing an intuitive view of S^4 in practice.

Synthetic Dataset. To construct the experimental dataset for composite provenance, a straightforward approach is to generate mixed-toolchain binaries by statically linking libraries compiled with different toolchains. However, this method is difficult to scale and unsuitable for quantitative analysis: Such binaries are affected by build configurations and link-time behaviors, which hinders controlled construction and introduces significant engineering overhead. In some cases, it is even impractical, as many real-world programs depend on components that are not readily amenable to static linking [14]. To enable quantitative and fine-grained evaluation, we construct a synthetic mixed-toolchain dataset via function-level composition. The BinKit dataset provides multiple compiler-specific realizations of the same program, compiled from identical source code with different compilers, which allows us to precisely control the contribution of each toolchain. Given a binary compiled with a host compiler C_h , we replace a contiguous region of functions in the binary address space with their corresponding realizations produced by a library compiler C_l for the same program. This design respects link-time locality [38, 60], whereby functions originating from the same object file or library are typically placed contiguously.

In cases where exact function correspondence is unavailable due to optimizations, we adopt a best-effort fallback by replacing a contiguous instruction region from the C_h binary with the corresponding sequence from the C_l realization. We vary the library contribution to 10%, 20%, 30%, 40%, and 50% of the binary to simulate different degrees of toolchain mixing.

Metrics. Let $\mathcal{D}$ denotes the mixed-toolchain dataset. Given a binary $x \in \mathcal{D}$ with ground truth

$$G(x) = \{C_h, C_l\},$$

where C_h and C_l denote the host and library compilers, respectively. S^4 produces two predictions through two distinct analysis paths: a primary prediction $C_{\text{primary}}(x)$ obtained from global analysis, and a secondary prediction $C_{\text{secondary}}(x)$ obtained from window-based analysis with majority voting (as described in Section 5). We denote the combined prediction set as

$$\text{Pred}(x) = \{C_{\text{primary}}(x), C_{\text{secondary}}(x)\}.$$

We then define

$$\text{Recall@}_{2\text{host/lib}} = \frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \mathbf{1}[C_{h/l} \in \text{Pred}(x)].$$

where $\mathbf{1}[\cdot]$ denotes the indicator function that equals 1 if the condition holds and 0 otherwise. Exact version attribution for library compiler can be overly restrictive in composite settings, as library-induced signals are spatially localized and often dominated by the global footprint of the host compiler. To capture partially resolved provenance under relaxed relations, we further define two metrics. *Library Coarse-Version Recall@2* groups compiler versions within each family into coarse buckets (e.g., gcc-{low, mid, high} and

clang-{low, mid, high}), and measures whether the predicted library compiler falls into the correct group. *Library Family Recall@2* evaluates whether the predicted library compiler belongs to the correct compiler family (e.g., gcc or clang). Both coarse-version and family recall are defined analogously to exact recall under their respective label abstractions. These metrics measure S^4 's ability to surface secondary provenance at progressively coarser resolutions. **Results.** Figure 4 summarizes the results across all four host-library configurations. Each sub-figure reports four metrics defined above as a function of the library code ratio.

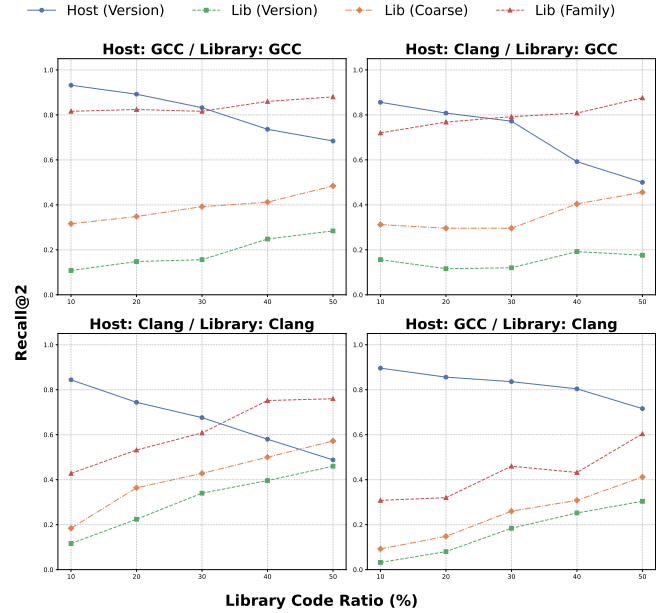


Figure 4: Composite provenance evaluation under the two-toolchain setting. Each subfigure reports Host Recall@2 along with Lib Recall@2 metrics at three granularities as the library code increases, across all host-library combinations.

Host Compiler. Across all configurations, S^4 consistently maintains high Host Recall@2 when the library footprint is small. At a 10% library ratio, Host Recall@2 ranges from 0.84 to 0.93 across different host-library combinations. As the library ratio increases, host recall gradually decreases, reflecting the growing influence of secondary compiler code. Nevertheless, even at a 50% library ratio, the primary compiler remains correctly surfaced in the majority of cases, with Host Recall@2 between 0.49 and 0.72 depending on the configuration. These results indicate that primary compiler identification is robust to substantial interference from library code produced by different toolchains.

Library Compiler (Exact Version). Exact version-level identification of the library compiler is significantly more challenging. Library Recall@2 remains low across all settings, particularly when the library contribution is small. At a 10% library ratio, version-level recall ranges from 0.03 to 0.16, and increases only modestly as the library ratio grows, reaching at most 0.46 at a 50% library ratio. This behavior is expected: secondary compiler signals are spatially

localized and easily overshadowed by the global stylistic patterns introduced by the host compiler, making fine-grained version attribution inherently challenging in composite settings.

Library Compiler (Coarse Version). When relaxing version granularity, S^4 exhibits substantially improved secondary provenance recovery. Library Coarse-Version Recall@2 consistently outperforms exact version recall across all configurations and increases steadily as the library footprint grows. Even at a 10% library ratio, coarse-version recall ranges from 0.18 to 0.32, indicating that S^4 can already surface partially resolved version-level signals from limited secondary code. As the library ratio increases to 40–50%, coarse-version recall reaches 0.41–0.57, depending on the host-library combination. Notably, configurations where the host and library compilers belong to the same family exhibit faster improvements, suggesting that once family identity is established, stylistic differences across version ranges become increasingly detectable. These results demonstrate that inference-time aggregation can recover meaningful intermediate-granularity provenance without retraining compiler signatures.

Library Compiler (Family). At the coarsest resolution, S^4 achieves strong and stable performance. Library Family Recall@2 is consistently high and improves monotonically with the library ratio. Across configurations, family-level recall often exceeds 0.70 at a 10% library ratio and reaches 0.76–0.88 when the library contribution grows to 40–50%. This confirms that compiler family lineage constitutes the most robust form of secondary provenance under composite settings, serving as an effective upper bound for finer-grained attribution.

Practical implications. The experimental results reveal a clear trade-off between attribution granularity and reliability in composite provenance. While exact version attribution for library compiler is not reliable due to limited signals, S^4 can reliably surface secondary provenance at coarser but practically meaningful resolutions. In real-world analysis, stable identification of the primary compiler provides a critical foundation for downstream tasks such as disassembly and decompilation. For secondary components, coarse-version and family-level provenance offer actionable insights for supply-chain auditing and security analysis, capturing toolchain lineage and evolutionary era without overconfident attribution. The ability of S^4 to recover such hierarchical provenance at inference time, without retraining, highlights its practical utility for mixed-toolchain binaries.

Mixed-Toolchain Binaries in Real-World Projects. To bridge the gap between synthetic experiment and real-world scenarios, we intervened in the compiling and linking processes and statically built six open-source projects (binutils [23], Gawk [22], coreutils [24], Redis [61], tmux [70], and xorriso [25]). Each project is built with two compilers, resulting in binaries that combine host code and statically linked library code compiled by different toolchains. Note that we only modify flags related to static linking and debug-symbol emission, keeping other flags (e.g., the optimization level) at their default values to reflect common settings. To obtain the coverage of different toolchains in a mixed-toolchain binary, we leverage the DWARF debugging information [20] emitted at compile time. Specifically, a binary can be modeled as a collection of compilation units (CUs) [20, 34]. Each CU has

a DW_AT_producer attribute, which encodes the compiler and a set of the invocation flags (e.g., GNU C11 11.2.0 [flags]), allowing us to attribute it to a specific toolchain. The address range of a CU is recovered from its DW_AT_low_pc, DW_AT_high_pc, and DW_AT_ranges attributes, which delimit the corresponding code region.

Because of the complexity of static builds and the combinatorial explosion of compiler configurations, exhaustively covering all toolchain combinations is generally impractical. Guided by our research question, we made two pragmatic design choices: 1) compiling the host code and library code with different compiler families; and 2) constructing cross-family toolchain pairs by randomly selecting one compiler from each GCC and Clang version-bucket (i.e., low, mid, high). This design yields $3 \times 3 \times 2 = 18$ toolchain combinations for each project, covering all GCC $\times$ Clang combinations in both host-library directions. Detailed statistics on the generated mixed-toolchain binaries are provided in Appendix C.

Before discussing the results, we note that host compiler and library compiler are not suitable labels in this case study, because library code often dominates host code for these fully statically-linked cases. Across all binaries, the average host share is 16.0%, compared with 69.5% for library code. This pattern is pronounced in projects composed of many small utilities, such as binutils and coreutils. Therefore, we report the results in terms of the *primary* and *secondary* compiler, which denote the larger and smaller code contributors, respectively.

Table 2: Secondary compiler Recall@2 stratified by the byte-level share of secondary code in each binary.

Secondary share	Recall@2 (secondary)		
	exact-version	coarse-version	family
< 5%	0.009	0.332	0.984
5–10%	0.383	0.700	1.000
10–20%	0.833	0.889	1.000
20–30%	0.639	0.861	1.000
30–60%	0.810	1.000	1.000

In this case study, our approach identifies the primary compiler reliably at every granularity (Recall@2 of 0.890 exact, 0.928 coarse-version, and 0.932 family). Secondary provenance depends strongly on the secondary code share. As shown in Table 2, when secondary-compiler code exceeds 10% of a binary, S^4 can surface meaningful exact and coarse-version signals. Moreover, it provides a strong family-level identification (Recall@2 = 0.984) even when the secondary code occupies less than 10%.

Taken together, this case study confirms that S^4 generalizes from the synthetic setting to real-world mixed-toolchain binaries. For a downstream analyst inspecting an unfamiliar statically linked binary, this is sufficient to attribute the bulk of the code to a specific compiler toolchain, to establish whether two toolchain families coexist in the same binary, and to localize the secondary toolchain to a coarse-version bucket whenever it accounts for a non-trivial share of the binary, yielding practically useful insights in the wild.

Answer to RQ4: S^4 reliably identifies the primary compiler and surfaces meaningful secondary provenance at multiple resolutions, ranging from family-level lineage to coarse version attribution. This behavior holds on real-world mixed-toolchain binaries.

7.6 RQ5: Probing Resilience

Probing Strategies. We consider two adaptive probing strategies. The first is based on Divide-and-Conquer (DC), which is widely observed in ITW evasion attacks and toolkits [49, 64, 66, 73]. This strategy recursively partitions the binary and probes subsets to isolate decisive regions that influence the provenance outcome. The second leverages Genetic Programming (GP), which has demonstrated strong effectiveness in black-box probing of security systems [2, 5, 7, 9–11, 41, 42, 53, 76]. GP evolves a population of candidate perturbations guided by fitness derived from the oracle’s outputs, enabling efficient exploration of the decision boundary under query and perturbation constraints. Appendix B further details these two probing strategies, including their principles and algorithms.

Dataset. Since the probing procedure is time-consuming due to a large number of iterative queries, we randomly select 10% of the test set consisting of binaries that are correctly attributed to their corresponding compiler labels.

Baselines. We compare S^4 and S^4 -MTD against the PassTell (best-performing) and all PM-based baselines (YarGen and AutoYara).

requiring a clearly larger number of iterations to achieve successful probing. This indicates that the MTD mechanism effectively disrupts the feedback loop of adversaries and increases the cost of exploration, thereby enhancing resilience against adaptive probing.

We further observe a clear difference between PM- and ML-based approaches. AutoYara and YarGen are considerably more vulnerable to adaptive probing than ML-based systems. In particular, YarGen maintains a success rate above 80% under DC-based probing. A manual inspection of successful attacks against YarGen reveals that all of them require only a single-byte modification. This behavior highlights a fundamental weakness of pattern matching techniques: their reliance on exact matching induces sharp decision boundaries, which can be efficiently identified by adaptive probing.

Finally, we compare the efficiency of two probing strategies. GP-based probing consistently requires more iterations than DC-based probing and achieves fewer successful probeings across all systems. We attribute this difference to the inherently stochastic nature of genetic programming, which introduces randomness into the search process and reduces the efficiency of boundary localization compared to the more structured divide-and-conquer strategy.

Answer to RQ5: Experiments show that the MTD mechanism effectively hinders adversarial probing by increasing probing cost and degrading probing effectiveness.

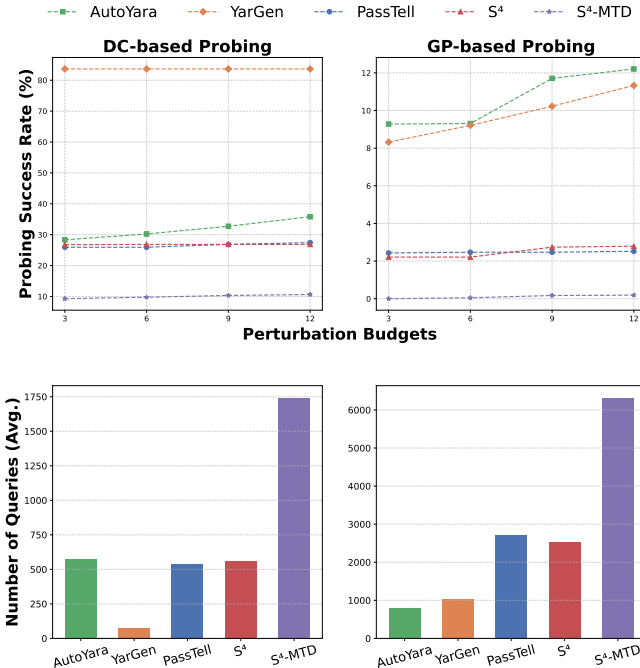


Figure 5: Evaluation on probing resilience against DC- and GP-based probing strategies.

Results. Figure 5 shows the results. Introducing MTD substantially improves robustness against adaptive probing attacks. Across both DC-based and GP-based probing strategies, S^4 -MTD consistently exhibits a significantly lower probing success rate than S^4 , while

7.7 RQ6: Explainable Decision

Methodology. Explainability is a derived property of S^4 , emerging naturally from the decoupled identification architecture. We evaluate explainability by inspecting feature contributions at the level of each compiler signature. Since S^4 models each compiler using an independent statistical signature, provenance decisions can be explained by examining the features that contribute to the output of the corresponding signature in isolation. We extract feature importance scores from each compiler signature using standard tree-based attribution methods and examine the most influential instruction patterns for both compiler identification and optimization-level identification.

Results. S^4 provides clear and localized explanations for provenance decisions. Since each statistical signature operates independently, the features contributing to a decision can be directly attributed to a specific compiler, without interference from unrelated classes. Appendix Table 6 presents representative feature importance results for compiler identification. For example, in identifying gcc-11.2.0, the most influential features involve the instruction endbr64, which is introduced as part of Intel CET. Since endbr64 also appears in earlier compiler versions, it is not sufficiently distinctive on its own. Instead, S^4 assigns high importance to composite instruction patterns in which endbr64 is immediately followed by instructions such as `jmp <NUM>`, `cmp <NUM>, <NUM>`, or `test <REG>, <REG>`. These paired patterns capture version-specific code generation behavior and provide interpretable evidence for compiler identification. Appendix Table 7 shows feature importance for optimization-level identification. A representative example is `Ofast`, whose most influential feature is the instruction `ldmxcsr [<REG>*<NUM>+<NUM>]`, a Streaming SIMD Extensions operation designed to accelerate floating-point computation. This result aligns

with the design goal of Ofast and demonstrates that S^4 captures semantically meaningful provenance patterns.

In contrast, explanations produced by monolithic models are less localized. Since features are shared across all classes, feature importance reflects global discrimination rather than compiler-specific evidence. Consequently, it is challenging to isolate the influence of other compiler classes, interpret the rationale behind a specific compiler prediction, or attribute decisions to concrete, compiler-specific artifacts.

Answer to RQ6: Decoupled identification architecture provides isolated and interpretable evidence for provenance decisions.

8 Discussion and Limitation

This work reframes compiler provenance as a continuously maintained service rather than a static classification task. Our experiments demonstrate that long-standing challenges, such as retraining-induced drift and feature entanglement, are fundamentally architectural in nature, rather than merely algorithmic. In this section, we clarify several important aspects of scope, limitations, and practicality to provide a complete picture of our work.

Scope of Compiler Provenance. Compiler provenance, as addressed in this work, focuses on identifying the original compilation toolchain used in standard compilation. Packer identification, which detects binary packers or obfuscators applied after standard compilation, is another task and lies beyond the scope of this study. In addition, another class of obfuscation techniques is compile-time obfuscation, which is typically based on LLVM (e.g., O-LLVM [39] and Hikari [68]) and transforms programs through obfuscation passes during compilation. Because these methods deliberately alter instruction structure (e.g., via instruction substitution or the introduction of spurious control flow), they may significantly affect regular compiler provenance that relies on code patterns. In such cases, alternative strategies may be required, e.g., analyzing provenance after deobfuscation [27, 45].

Scope of Probing Resilience. Our MTD mechanism exclusively defends against the critical feature localization of black-box evasion attacks. By limiting the exposure of stable features, it significantly increases the query cost required for adaptive probing. While this significantly raises the bar for successful probing, it does not address the subsequent stage of translating inferred feature knowledge into concrete, semantics-preserving binary transformations (e.g., instruction substitution, dead code insertion, or layout randomization). These follow-up steps for crafting evasion remain beyond the scope of this work.

Practical Scalability of Signatures. Although decoupled identification introduces one signature per compiler, the overhead is well-contained in the compiler provenance domain because the number of relevant classes remains stable and predictable. For instance, GCC has released 14 major versions (59 minor) over 37 years [26], and Clang has released 18 major versions (47 minor) over 21 years [46]. This bounded growth ensures that the overhead of maintaining independent signatures remains manageable. A strong precedent for this design can be found in widely-used pattern matching tools such as PEiD [1], Exeinfo PE [4], and DIE [54]. These tools maintain an independent signature for each known

compiler or binary pack, with databases routinely containing thousands of entries. Despite this per-component signature approach, these tools have remained practical and actively maintained for decades in the malware analysis and reverse engineering communities, demonstrating that independent per-compiler signatures impose no prohibitive scalability or maintenance burden in real-world scenarios.

Extend to Multiple Secondary Compilers. Our experiments instantiate the composite provenance in the common two-toolchain setting. However, our method described in Section 5 can potentially be extended to binaries containing code from multiple secondary compilers (e.g., a host program statically linked with libraries produced by two or more distinct toolchains). Formally, let $\mathcal{W}$ denote the set of sliding windows over the input binary B , and let $\hat{C}(w)$ be the temporal-priority winner produced by the signature bank on window w . As described in Section 5, after identifying the host compiler C_h via global analysis, we discard host-dominant windows and define the residual set $\mathcal{W}_r = \{w \in \mathcal{W} \mid \hat{C}(w) \neq C_h\}$. Over $\mathcal{W}_r$, we compute per-compiler vote counts $v(C_i) = |\{w \in \mathcal{W}_r \mid \hat{C}(w) = C_i\}|$ and report the top- k secondary compilers ranked by $v(C_i)$, with temporal priority breaking ties. The previous two-toolchain setting in our experiment corresponds to $k = 1$, which assumes one secondary compiler. In addition, link-time locality [38, 60] can be further explored under these settings. For example, contiguous runs of windows sharing the same $\hat{C}(w)$ can be merged into segments before voting, strengthening signals from genuinely contiguous library regions and reducing sensitivity to isolated false positives. A systematic evaluation of compositions involving more than two toolchains is left to future work, as such cases require a separate evaluation design and may be less common in practice than the two-toolchain setting considered in this paper.

9 Related Work

Practical Tools for Compiler Provenance. Compiler provenance is widely used as a prerequisite in binary analysis and reverse engineering. Mainstream tools such as IDA Pro [32] and Ghidra [52] require users to specify the compiler specification as prior knowledge to facilitate disassembly and decompilation. In practice, compiler provenance is often performed by signature-based tools that rely on syntactic pattern matching over binaries and metadata. Representative tools include PEiD [1], Exeinfo PE [4], and Detect It Easy (DIE) [54]. These tools typically couple a pattern-matching engine with a user-editable signature database, which makes them efficient, explainable, and easy to extend. However, their reliance on superficial indicators also makes them fragile in adversarial settings. For instance, GCC always appends a magic string (e.g., “GCC: (Debian 10.2.0-5) 10.2.0”) into the `.comment` section to indicate the original compiler. These distinctive marks are convenient for signature matching but can be easily removed, modified, or forged. Such weaknesses have been observed in the wild, including malware that forges Rich header information [40] or strips provenance-related symbols and sections to hinder analysis [50, 51]. Related work in automated signature generation (e.g., YarGen [21] and AutoYara [58]) further improves usability, but the derived rules remain syntactic and inherit similar manipulation risks.

Statistical Compiler Provenance. Academic research has largely formulated compiler provenance as a multiclass classification problem. Prior studies extract provenance-related features from raw bytes [31], instructions [13, 18, 19, 44, 55, 63, 69, 77], structural information such as call or control-flow relationships [6, 36, 44, 48, 59, 62], and execution traces [18, 19]. Compared to syntactic pattern matching, statistical approaches aim to capture statistical patterns in generated code and are less dependent on shallow metadata. Most works demonstrate improvements by adopting increasingly complex models, including deep neural networks [6, 12, 13, 35, 36, 55, 69, 77]. In contrast, our work is motivated by deployment-oriented requirements that are underexplored in prior studies, including continuous compiler evolution, maintenance without global retraining, and explicit handling of composite provenance. Rather than pursuing accuracy primarily through model complexity, we focus on system design and decision semantics for CPaaS settings, while remaining compatible with a broad range of feature representations and learning algorithms.

Adaptive Probing Attacks. The reliability of provenance systems is increasingly challenged in adversarial environments. ITW observations indicate that attackers actively manipulate provenance-related artifacts, including forging metadata and stripping symbols or sections [40, 50, 51]. More broadly, security research has established that black-box systems with deterministic feature extraction can be vulnerable to adaptive probing, where attackers iteratively query a detector and localize critical decision cues. Such adaptive probing has been widely studied in malware evasion [15, 16, 49, 56, 64, 66, 73]. These attacks exploit response consistency across queries and can be instantiated with heuristic procedures such as divide-and-conquer localization or evolutionary search. Our work draws on this line of research to model adaptive probing against compiler provenance systems.

10 Conclusion

We argue that compiler provenance should be treated as a continuously maintained service. We present S^4 , which decouples provenance into per-compiler statistical signatures and resolves evidence with a temporal-priority policy. This design preserves identification accuracy while providing operational guarantees that monolithic models struggle to offer: low drift under incremental compiler integration, localized refinement without system-wide regressions, and composable evidence for mixed-toolchain binaries. We also propose an optional moving target defense based on randomized featurization that increases the cost and uncertainty of adaptive probing with minimal impact on usability. Together, these results demonstrate that deployment-ready provenance requires architectural decoupling and explicit decision semantics, not only stronger classifiers.

11 Ethical Considerations

No ethical issues have been identified in this research.

Acknowledges

This material is based upon work supported by the National Science Foundation under grant No. 2145744, No. 2229876, and No. 2442339 and is supported in part by funds provided by the National Science

Foundation (NSF), by the Department of Homeland Security, by the Defense Advanced Research Projects Agency (DARPA) under contract number N6600120C4031, and by IBM. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the NSF or its federal agency and industry partners.

References

- [1] ALDEID. 2026. PEiD - aldeid. <https://www.aldeid.com/wiki/PEiD>.
- [2] Moustafa Alzantot, Yash Sharma, Supriyo Chakraborty, Huan Zhang, Cho-Jui Hsieh, and Mani B. Srivastava. 2019. GenAttack: practical black-box attacks with gradient-free optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '19)*. Association for Computing Machinery, New York, NY, USA, 1111–1119. doi:10.1145/3321707.3321749
- [3] Anonymous. 2026. S4. <https://anonymous.4open.science/r/S4-51B4/>.
- [4] A.S.L. 2026. Exeinfo PE. <http://www.exeinfo.byethost18.com/>.
- [5] Emre Aydogan and Sevil Sen. 2015. Automatic Generation of Mobile Malwares Using Genetic Programming. In *Applications of Evolutionary Computation*, Antonio M. Mora and Giovanni Squillero (Eds.). Springer International Publishing, Cham, 745–756.
- [6] Tristan Benoit, Jean-Yves Marion, and Sébastien Bardin. 2021. Binary level toolchain provenance identification with graph neural networks. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 131–141. doi:10.1109/SANER50967.2021.00021
- [7] Kevin Bock, George Hughey, Xiao Qiang, and Dave Levin. 2019. Geneva: Evolving Censorship Evasion Strategies. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (London, United Kingdom) (CCS '19)*. Association for Computing Machinery, New York, NY, USA, 2199–2214. doi:10.1145/3319535.3363189
- [8] Michael D. Brown, Matthew Pruett, Robert Bigelow, Girish Mururu, and Santosh Pande. 2021. Not so Fast: Understanding and Mitigating Negative Impacts of Compiler Optimizations on Code Reuse Gadget Sets. *Proc. ACM Program. Lang.* 5, OOPSLA, Article 154 (oct 2021), 30 pages. doi:10.1145/3485531
- [9] Alejandro Calleja, Alejandro Martín, Héctor D. Menéndez, Juan Tapiador, and David Clark. 2018. Picking on the family: Disrupting android malware triage by forcing misclassification. *Expert Systems with Applications* 95 (2018), 113–126. doi:10.1016/j.eswa.2017.11.032
- [10] Raphael Labaca Castro, Corinna Schmitt, and Gabi Dreier. 2019. AIMED: Evolving Malware with Genetic Programming to Evade Detection. In *2019 18th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*. 240–247. doi:10.1109/TrustCom/BigDataSE.2019.00040
- [11] Jinyin Chen, Mengmeng Su, Shijing Shen, Hui Xiong, and Haibin Zheng. 2019. POBA-GA: Perturbation optimized black-box adversarial attacks via genetic algorithm. *Computers & Security* 85 (2019), 89–106. doi:10.1016/j.cose.2019.04.014
- [12] Ligeng Chen, Zhongling He, Hao Wu, Fengyuan Xu, Yi Qian, and Bing Mao. 2022. DiComp: Lightweight Data-Driven Inference of Binary Compiler Provenance with High Accuracy. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 112–122. doi:10.1109/SANER53432.2022.00025
- [13] Yu Chen, Zhiqiang Shi, Hong Li, Weiwei Zhao, Yiliang Liu, and Yuansong Qiao. 2019. HIMALIA: Recovering Compiler Optimization Levels from Binaries by Deep Learning. In *Intelligent Systems and Applications*, Kohei Arai, Supriya Kapoor, and Rahul Bhatia (Eds.). Springer International Publishing, Cham, 35–47.
- [14] Will Dietz and Vikram Adve. 2018. Software multiplexing: share your libraries and statically link them too. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 154 (Oct. 2018), 26 pages. doi:10.1145/3276524
- [15] Dobin Rutishauser. 2023. Cracking the Shield: Analyzing and Reverse Engineering Antivirus Signatures. <https://conference.hitb.org/hitbsecconf2023hkt/session/cracking-the-shield-analyzing-and-reverse-engineering-antivirus-signatures/>.
- [16] Dobin Rutishauser. 2026. avred. <https://github.com/dobin/avred>.
- [17] Vijay D'Silva, Mathias Payer, and Dawn Song. 2015. The Correctness-Security Gap in Compiler Optimization. In *2015 IEEE Security and Privacy Workshops*. 73–87. doi:10.1109/SPW.2015.33
- [18] Yufei Du, Omar Alrawi, Kevin Snow, Manos Antonakakis, and Fabian Monrose. 2023. Improving Security Tasks Using Compiler Provenance Information Recovered At the Binary-Level. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. Association for Computing Machinery, New York, NY, USA, 2695–2709. doi:10.1145/3576915.3623098
- [19] Yufei Du, Ryan Court, Kevin Snow, and Fabian Monrose. 2022. Automatic Recovery of Fine-grained Compiler Artifacts at the Binary Level. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 853–868. <https://www.usenix.org/conference/atc22/presentation/du>
- [20] DWARF Debugging Information Format Committee. 2026. DWARF Debugging Information Format. <https://dwarfstd.org/>.

- [21] Florian Roth. 2026. Yara Rule Generator. <https://github.com/Neo23x0/yarGen>.
- [22] Free Software Foundation. 2023. Gawk: GNU awk. <https://www.gnu.org/software/gawk/>.
- [23] Free Software Foundation. 2026. GNU Binutils. <https://www.gnu.org/software/binutils/>.
- [24] Free Software Foundation. 2026. GNU Coreutils. <https://www.gnu.org/software/coreutils/>.
- [25] Free Software Foundation. 2026. GNU xorriso. <https://www.gnu.org/software/xorriso/>.
- [26] Free Software Foundation, Inc. 2026. GCC Releases - GNU Project. <https://gcc.gnu.org/releases.html>.
- [27] Francis Gabriel. 2014. Deobfuscation: recovering an OLLVM-protected program. <https://blog.quarkslab.com/deobfuscation-recovering-an-llvm-protected-program.html>.
- [28] Lian Gao, Yu Qu, Sheng Yu, Yue Duan, and Heng Yin. 2024. SigmaDiff: Semantics-Aware Deep Graph Matching for Pseudocode Diffing. In *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/sigmadiff-semantics-aware-deep-graph-matching-for-pseudocode-diffing/>.
- [29] GNU Project. 2026. Using as: The GNU Assembler. <https://sourceware.org/binutils/docs/as/>.
- [30] Trevor Hastie, Robert Tibshirani, and Jerome H. Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd Edition*. Springer. doi:10.1007/978-0-387-84858-7
- [31] Xu He, Shu Wang, Yunlong Xing, Pengbin Feng, Haining Wang, Qi Li, Songqing Chen, and Kun Sun. 2022. BinProv: Binary Code Provenance Identification without Disassembly. In *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses (Limassol, Cyprus) (RAID '22)*. Association for Computing Machinery, New York, NY, USA, 350–363. doi:10.1145/3545948.3545956
- [32] Hex-Rays SA. 2026. IDA Pro. <https://hex-rays.com/ida-pro/>.
- [33] Michael J. Hohnka, Jodi A. Miller, Kenrick M. Dacumos, Timothy J. Fritton, Julia D. Erdley, and Lyle N. Long. 2019. Evaluation of Compiler-Induced Vulnerabilities. *Journal of Aerospace Information Systems* 16, 10 (2019), 409–426. doi:10.2514/1.1010699 arXiv:https://doi.org/10.2514/1.1010699
- [34] International Organization for Standardization. 2018. ISO/IEC 9899:2018: Information technology — Programming languages — C. <https://www.iso.org/standard/74528.html>.
- [35] Maliha Ismail, Yan Lin, DongGyun Han, and Debin Gao. 2023. BinAlign: Alignment Padding Based Compiler Provenance Recovery. In *Information Security and Privacy, Leonie Simpson and Mir Ali Rezazadeh Bae (Eds.)*. Springer Nature Switzerland, Cham, 609–629.
- [36] Yuede Ji, Lei Cui, and H. Howie Huang. 2021. Vestige: Identifying Binary Code Provenance for Vulnerability Detection. In *Applied Cryptography and Network Security, Kazue Sako and Nils Ole Tippenhauer (Eds.)*. Springer International Publishing, Cham, 287–310.
- [37] Lichen Jia, Chenggang Wu, Bowen Tang, Peihua Zhang, Zihan Jiang, Yang Yang, Ning Liu, Jingfeng Zhang, and Zhe Wang. 2024. Enhancing Learning-Based Binary Code Similarity Detection Model through Adversarial Training with Multiple Function Variants. In *Findings of the Association for Computational Linguistics: EMNLP 2024, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.)*. Association for Computational Linguistics, Miami, Florida, USA, 11508–11518. doi:10.18653/v1/2024.findings-emnlp.673
- [38] Ling Jiang, Junwen An, Huihui Huang, Qiyi Tang, Sen Nie, Shi Wu, and Yuqun Zhang. 2024. BinaryAI: Binary Software Composition Analysis via Intelligent Binary Source Code Matching. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 224, 13 pages. doi:10.1145/3597503.3639100
- [39] Pascal Junod, Julien Rinaldini, Johan Wehrli, and Julie Michielin. 2015. Obfuscator-LLVM – Software Protection for the Masses. In *Proceedings of the IEEE/ACM 1st International Workshop on Software Protection, SPRO'15, Firenze, Italy, May 19th, 2015, Brecht Wyseur (Ed.)*. IEEE, 3–9. doi:10.1109/SPRO.2015.10
- [40] Kaspersky Lab. 2018. The devil's in the Rich header. <https://securelist.com/the-devils-in-the-rich-header/84348/>.
- [41] Hilmi Güneş Kayacik, Malcolm Heywood, and Nur Zincir-Heywood. 2006. On evolving buffer overflow attacks using genetic programming. In *Proceedings of the 8th Annual Conference on Genetic and Evolutionary Computation (Seattle, Washington, USA) (GECCO '06)*. Association for Computing Machinery, New York, NY, USA, 1667–1674. doi:10.1145/1143997.1144271
- [42] Hilmi Güneş Kayacik, Nur Zincir-Heywood, and Malcolm I. Heywood. 2011. Evolutionary computation as an artificial attacker: generating evasion attacks for detector vulnerability testing. *Evol. Intell.* 4, 4 (2011), 243–266. doi:10.1007/S12065-011-0065-0
- [43] Dongkwan Kim, Eunsoo Kim, Sang Kil Cha, Soel Son, and Yongdae Kim. 2023. Revisiting Binary Code Similarity Analysis Using Interpretable Feature Engineering and Lessons Learned. *IEEE Trans. Softw. Eng.* 49, 4 (April 2023), 1661–1682. doi:10.1109/TSE.2022.3187689
- [44] Jason Kim, Daniel Genkin, and Kevin Leach. 2023. Revisiting Lightweight Compiler Provenance Recovery on ARM Binaries. In *2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*. 292–303. doi:10.1109/ICPC58990.2023.00044
- [45] Yuhao Li, Bin Wen, and Haixiao Zheng. 2023. Generic O-LLVM Automatic Multi-architecture Deobfuscation Framework Based on Symbolic Execution. In *Proceedings of the 4th International Conference on Advanced Information Science and System (Sanya, China) (AISS '22)*. Association for Computing Machinery, New York, NY, USA, Article 59, 6 pages. doi:10.1145/3573834.3574541
- [46] LLVM Project. 2026. Download LLVM releases. <https://releases.llvm.org/>.
- [47] Kangjie Lu, Chengyu Song, Taesoo Kim, and Wenke Lee. 2016. UniSan: Proactive Kernel Memory Initialization to Eliminate Data Leakages. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (Vienna, Austria) (CCS '16)*. Association for Computing Machinery, New York, NY, USA, 920–932. doi:10.1145/2976749.2978366
- [48] Luca Massarelli, Giuseppe A Di Luna, Fabio Petroni, Leonardo Querzoni, Roberto Baldoni, et al. 2019. Investigating graph embedding neural networks with unsupervised features extraction for binary analysis. In *Proceedings of the 2nd Workshop on Binary Analysis Research (BAR)*. 1–11.
- [49] Matt Hand. 2026. DefenderCheck. <https://github.com/matterpreter/DefenderCheck>.
- [50] Mirai. 2026. Leaked Mirai Source Code for Research/IoC Development Purposes. <https://github.com/jgambin/Mirai-Source-Code>.
- [51] Mirai. 2026. Mirai Botnet Client, Echo Loader and CNC source code. <https://github.com/soufianetahiri/Mirai-Botnet/blob/295be2b055da29ca3f552b82d5b2580800db776/mirai/build.sh#L7>.
- [52] National Security Agency. 2026. Ghidra. <https://ghidra-sre.org/>.
- [53] Sadia Noreen, Shafaq Murtaza, M. Zubair Shafiq, and Muddassar Farooq. 2009. Evolvable malware. In *Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation (Montreal, Québec, Canada) (GECCO '09)*. Association for Computing Machinery, New York, NY, USA, 1569–1576. doi:10.1145/1569901.1570111
- [54] NTInfo. 2026. Detect It Easy. <http://ntinfo.biz/>.
- [55] Yuhei Otsubo, Akira Otsuka, Mamoru Mimura, Takeshi Sakaki, and Hiroshi Ukegawa. 2020. o-glassesX: Compiler Provenance Recovery with Attention Mechanism from a Short Code Fragment. *Proceedings 2020 Workshop on Binary Analysis Research* (2020).
- [56] R. Perdisci, D. Dagon, Wenke Lee, P. Fogla, and M. Sharif. 2006. Misleading worm signature generators using deliberate noise injection. In *2006 IEEE Symposium on Security and Privacy (S&P'06)*. 15 pp.–31. doi:10.1109/SP.2006.26
- [57] Davide Pizzolotto and Katsuro Inoue. 2020. Identifying Compiler and Optimization Options from Binary Code using Deep Learning Approaches. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 232–242. doi:10.1109/ICSME46990.2020.00031
- [58] Edward Raff, Richard Zak, Gary Lopez Munoz, William Fleming, Hyrum S. Anderson, Bobby Filar, Charles Nicholas, and James Holt. 2020. Automatic Yara Rule Generation Using Biclustering. In *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security (Virtual Event, USA) (AISec'20)*. Association for Computing Machinery, New York, NY, USA, 71–82. doi:10.1145/3411508.3421372
- [59] Ashkan Rahimian, Paria Shirani, Saeed Alrbaee, Lingyu Wang, and Mourad Debabi. 2015. BinComp: A stratified approach to compiler provenance Attribution. *Digital Investigation* 14 (2015), S146–S155. doi:10.1016/j.diin.2015.05.015 The Proceedings of the Fifteenth Annual DFRWS Conference.
- [60] REcon. 2018. Recovering Software Architecture from Embedded Binaries. <https://recon.cx/2018/montreal/schedule/events/109.html>.
- [61] Redis Ltd. 2026. Redis. <https://github.com/redis/redis>.
- [62] Nathan Rosenblum, Barton P. Miller, and Xiaojin Zhu. 2011. Recovering the Toolchain Provenance of Binary Code. In *Proceedings of the 2011 International Symposium on Software Testing and Analysis (Toronto, Ontario, Canada) (ISSTA '11)*. Association for Computing Machinery, New York, NY, USA, 100–110. doi:10.1145/2001420.2001433
- [63] Nathan E. Rosenblum, Barton P. Miller, and Xiaojin Zhu. 2010. Extracting Compiler Provenance from Program Binaries. In *Proceedings of the 9th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (Toronto, Ontario, Canada) (PASTE '10)*. Association for Computing Machinery, New York, NY, USA, 21–28. doi:10.1145/1806672.1806678
- [64] RythmStick. 2026. The Hunt for Malicious Strings. <https://github.com/RythmStick/AMSTrigger>.
- [65] scikit-learn developers. 2026. OneVsRestClassifier — scikit-learn documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.multiclass.OneVsRestClassifier.html>.
- [66] SCRT. 2026. avdebugger. <https://github.com/scrt/avdebugger>.
- [67] Laurent Simon, David Chisnall, and Ross Anderson. 2018. What You Get is What You C: Controlling Side Effects in Mainstream C Compilers. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. 1–15. doi:10.1109/EuroSP.2018.00009

- [68] The YaraRules team. 2026. Hikari LLVM-based Obfuscator. <https://github.com/HikariObfuscator/Hikari>.
- [69] Zhenzhou Tian, Yaqian Huang, Borun Xie, Yanping Chen, Lingwei Chen, and Dinghao Wu. 2021. Fine-Grained Compiler Identification With Sequence-Oriented Neural Modeling. *IEEE Access* 9 (2021), 49160–49175. doi:10.1109/ACCESS.2021.3069227
- [70] tmux contributors. 2026. tmux. <https://github.com/tmux/tmux>.
- [71] U.S. Department of Homeland Security. 2024. Moving Target Defense. <https://www.dhs.gov/archive/science-and-technology/csd-mtd>.
- [72] Xi Wang, Nickolai Zeldovich, M. Frans Kaashoek, and Armando Solar-Lezama. 2013. Towards Optimization-Safe Systems: Analyzing the Impact of Undefined Behavior. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles* (Farmington, Pennsylvania) (SOSP '13). Association for Computing Machinery, New York, NY, USA, 260–275. doi:10.1145/2517349.2522728
- [73] Christian Wressnegger, Kevin Freeman, Fabian Yamaguchi, and Konrad Rieck. 2017. Automatically Inferring Malware Signatures for Anti-Virus Assisted Attacks. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security* (Abu Dhabi, United Arab Emirates) (ASIA CCS '17). Association for Computing Machinery, New York, NY, USA, 587–598. doi:10.1145/3052973.3053002
- [74] Claud Xiao. 2015. Novel Malware XcodeGhost Modifies Xcode, Infects Apple iOS Apps and Hits App Store. <https://unit42.paloaltonetworks.com/novel-malware-xcodeghost-modifies-xcode-infects-apple-ios-apps-and-hits-app-store/>.
- [75] Jianhao Xu, Kangjie Lu, Zhengjie Du, Zhu Ding, Linke Li, Qiushi Wu, Mathias Payer, and Bing Mao. 2023. Silent Bugs Matter: A Study of Compiler-Introduced Security Bugs. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 3655–3672. <https://www.usenix.org/conference/usenixsecurity23/presentation/xu-jianhao>
- [76] Weilin Xu, Yanjun Qi, and David Evans. 2016. Automatically Evading Classifiers: A Case Study on PDF Malware Classifiers. In *23rd Annual Network and Distributed System Security Symposium, NDSS 2016, San Diego, California, USA, February 21–24, 2016*. The Internet Society. <http://wp.internetsociety.org/ndss/wp-content/uploads/sites/25/2017/09/automatically-evading-classifiers.pdf>
- [77] Shouguo Yang, Zhiqiang Shi, Guodong Zhang, Mingxuan Li, Yuan Ma, and Limin Sun. 2019. Understand Code Style: Efficient CNN-Based Compiler Optimization Recognition System. In *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*, 1–6. doi:10.1109/ICC.2019.8761073
- [78] Zhaomo Yang, Brian Johannesmeyer, Anders Trier Olesen, Sorin Lerner, and Kirill Levchenko. 2017. Dead Store Elimination (Still) Considered Harmful. In *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, Vancouver, BC, 1025–1040. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/yang>
- [79] yotsubo. 2026. o-glassesX. <https://github.com/yotsubo/o-glassesX>.

A Optimization-Level Identification

Optimization identification is a six-class classification task (O0–3, O5, Ofast). We merge the same optimization levels that come from GCC and Clang for clarity.

Table 3: Evaluation on Optimization-Level Identification

Category	Model	P (%)	R (%)	F (%)	Acc (%)
PM based	YarGen [21]	99.59	10.26	18.41	85.04
	AutoYara [58]	17.89	85.76	29.44	31.27
ML based	ORIGIN [62]	68.82	70.58	68.73	70.63
	O-glassesX (16) [55]	75.48	64.87	64.65	64.86
	O-glassesX (64) [55]	54.26	59.01	55.46	59.01
	DICoMP [12]	90.94	90.67	90.56	90.67
	DICoMP++ [44]	89.66	89.28	89.38	89.21
	BinAlign [35]	87.44	86.63	86.98	77.93
	PassTell [18, 19]	94.80	94.79	94.79	94.79
	S ⁴	94.81	94.82	94.81	94.82
	S ⁴ -MTD	94.22	94.09	94.14	94.09

Table 3 displays the results. Overall, S⁴-series models perform better than ML- and PM-based baselines. In terms of the PM-based approaches, YarGen and AutoYara reveal contrasting behaviors. YarGen attains the highest precision but the lowest recall and F1 score, indicating a high rate of false negatives. While its positive predictions are highly accurate, it fails to detect many actual positives, reflecting an overly conservative prediction strategy. In contrast, AutoYara exhibits high recall but low precision, which indicates a high rate of false positives. These results highlight that optimization identification requires statistical pattern, where syntactic-level pattern matching is largely ineffective.

Table 4: Identifying Adjacent Optimization Levels

	O0/O1	O1/O2	O2/O3	Os/O2	Ofast/O3
Acc (%)	97.45	90.39	87.16	91.67	94.22

In addition, existing binary-related studies suggest that distinguishing adjacent optimization levels is challenging (e.g., distinguishing O1 from O2, or O2 from O3) [28, 31, 37, 43]. Our experiments further investigate this issue. Table 4 presents the experimental results. Five combinations are evaluated³. The difficulty increases with higher optimization levels: O0/O1 achieves the highest accuracy at 97.45%, whereas O2/O3 is the hardest to distinguish, with 87.16% accuracy. This aligns with existing studies [37, 43] and confirms that differentiating O2 from O3 is challenging.

B Adaptive Probing Strategies

B.1 Probing based on Divide-and-Conquer

Divide-and-Conquer (DC) method is widely used in ITW evasion attacks [49, 64, 66, 73]. We treat a sample as an instruction sequence $I = [\iota_1, \dots, \iota_K]$ and query the detector via $scan(I)$. Starting from

³The documentations of GCC and Clang state that Os derives from O2, and Ofast from O3; thus we compare Os against O2, and Ofast against O3.

a known positive sample, DC first splits I into two instruction intervals. For an interval, we apply a feature-space masking operator $patch(I, interval)$ that replaces all instructions within the interval by a predefined mask representation (i.e., NOP). If masking an interval does not change the scan result, the interval is considered irrelevant and removed. Otherwise, the interval is recursively subdivided until the granularity constraint is met.

Algorithm 1 Localizing Adversarial Perturbations with DC in Instruction Space

Require: I : input instruction sequence; c : constraint

Ensure: ams : a set of adversarial instruction intervals

```

1:  $y \leftarrow scan(I)$ 
2:  $worklist \leftarrow split(I)$ 
3:  $ams \leftarrow \{\}$ 
4: while  $worklist$  is not empty do
5:    $interval \leftarrow worklist.pop()$ 
6:    $I' \leftarrow patch(I, interval)$ 
7:   if  $scan(I') \neq y$  then
8:     if  $length(interval) \leq c$  then
9:        $ams \leftarrow ams \cup \{interval\}$ 
10:    else
11:       $worklist.extend(split(interval))$ 
12:    end if
13:  end if
14: end while

```

We define the query budget Q as the total number of calls to $scan(\cdot)$ during one probing attempt. The perturbation budget P is the length of $interval$, i.e., constraint c .

B.2 Probing based on Genetic Programming

Genetic Programming (GP) is a widely used and effective technique for evading security systems under a black-box setting [2, 5, 7, 9–11, 41, 42, 53, 76]. GP is inspired by biological evolution and evolves candidate perturbations to satisfy an evasion objective. The search is driven by iterative refinement of a population using three standard operations: selection, crossover, and mutation. Selection chooses candidates according to a fitness function that quantifies progress toward the evasion goal. Crossover combines two selected candidates to produce offspring, which promotes exploration by recombining effective patterns. Mutation randomly alters candidates, which expands coverage of the search space and mitigates premature convergence.

In feature space, we represent a binary as an instruction sequence

$$I = [\iota_1, \dots, \iota_K],$$

and define an instruction-level masking vector

$$M = [m_1, \dots, m_K], \quad m_j \in \{0, 1\}.$$

Here, $m_j = 0$ keeps ι_j unchanged, and $m_j = 1$ masks ι_j in feature space. Let $patch(I, M)$ denote the masked instruction sequence obtained by replacing each ι_j with a predefined mask representation whenever $m_j = 1$. The detector is queried through $scan(\cdot)$, which takes an instruction sequence representation as input.

Table 5: Statistics on the two-toolchain binaries. The primary compiler of a given binary is whichever of gcc/clang contributes the larger byte share; the secondary is the other. Avg is two-stage (mean across all the toolchain pairs per binary, then mean across binaries). Min/Mid/Max are the minimum, median, and maximum byte-share percentages over all binaries of a project.

Project	# of Bin $\times$ Pairs		Avg (%)	Min (%)	Mid (%)	Max (%)
binutils-2.40	15×18	P	79.09	54.68	82.86	85.50
		S	6.33	0.39	2.32	28.98
coreutils-9.1	103×18	P	71.04	64.61	71.67	75.99
		S	1.18	0.08	0.69	5.83
gawk-5.2.1	1×18	P	58.40	57.88	58.22	59.17
		S	26.69	25.83	27.00	27.25
redis-5.0.0	6×18	P	58.19	45.35	55.31	78.66
		S	29.64	4.44	35.05	39.26
tmux-3.0	1×18	P	55.09	54.85	55.11	55.29
		S	28.38	28.19	28.37	28.58
xorriso-1.5.4	1×18	P	58.96	58.76	58.96	59.15
		S	29.29	29.09	29.30	29.47

Following prior GP-based evasion studies, we define the fitness function as

$$fitness = is_evade(I, M) + \frac{1}{sum(M) + 1},$$

where $is_evade(I, M) = 1$ if $scan(patch(I, M)) \neq scan(I)$ and 0 otherwise. The term $sum(M)$ counts the number of masked instructions. A value $fitness \geq 1$ indicates a successful evasion.

The evolution is implemented in two stages. Stage 1 aims to find a valid adversarial mask. The initial population is a set of all-zero vectors, which enforces a gradual increase in the number of masked instructions from zero. Once a valid mask M is found, stage 2 aims to reduce the perturbation while preserving validity, which corresponds to decreasing $sum(M)$ while keeping $is_evade(I, M) =$

1. In Stage 2, the initial population is sampled from a uniform distribution over $\{0, 1\}^K$. Both stages use the same fitness function and differ only in the initialization strategy.

The algorithm terminates when the best fitness value does not change for 20 generations.

C Mixed-Toolchain Binaries used in Case Study

We built six open-source projects in controlled environments and used the generated mixed-toolchain binaries in the case study presented in Section 7.5. Table 5 shows the statistics. Each project occupies two rows: row P reports statistics for the *primary* compiler—whichever of gcc/clang contributes the larger byte share in a given binary—and row S for the *secondary* compiler. # of Bin $\times$ Pairs is the number of distinct binaries built per project; every binary is built under all gcc $\times$ clang toolchain pairs. Avg is a two-stage mean: first across all the toolchain pairs of a binary, then across the binaries of the project, so every binary contributes equally regardless of project size. Min, Mid, and Max are the minimum, median, and maximum of the percentage taken over all binaries of the project, capturing the spread that the average hides; the primary share in redis-5.0.0, for instance, ranges from 45.35% to 78.66% even though its average is 58.19%. Note that the primary and secondary percentages typically do not sum to 100% because each binary additionally contains object code from the GNU assembler as [29] (e.g., startup files, libc primitives), and other .s sources that are attributable to neither gcc nor clang. This assembler share ranges from roughly 11.8% to 27.8% across the six projects.

In summary, the generated mixed-toolchain binaries contain varying proportions of code produced by the primary and secondary compilers, making these projects a representative case study.

D Open Science

We release the code and data at [3]. The repository provides step-by-step instructions for constructing the experimental datasets from BinKit and scripts to reproduce all experiments.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

Table 6: Top-3 Important Features for Compiler Identification

Compiler	Top-3 Feature
clang-4.0	mov <REG>,<REG> push [<REG>+<REG>*<NUM>+<NUM>] inc <REG> mov [<REG>],<REG> nop [<REG>+<REG>*<NUM>+<NUM>] inc <REG>
clang-5.0	movups <REG>,<REG>*<NUM>+<NUM> movups <REG>,<REG> mov [<REG>+<REG>*<NUM>+<NUM>],<REG> inc [<REG>] lea <REG>,<REG>+<REG>*<NUM>+<NUM> inc <REG>
clang-6.0	sub <REG>,<NUM> movups <REG>,<NUM> movups <REG>,<REG> movups <REG>,<REG>+<REG>*<NUM>+<NUM> cmp <REG>,<REG> mov <NUM>,<REG>
clang-7.0	mov <REG>,<REG> idiv <REG> movsx <REG>,<REG>+<REG>*<NUM>+<NUM> mov <REG>,<NUM> sub <REG>,<REG>*<NUM>+<NUM> <INDCALL>
clang-8.0	neg <REG> and <REG>,<REG>*<NUM>+<NUM> nop [<REG>] movups <REG>,<REG>+<REG>*<NUM>+<NUM> mov [<REG>],<REG> movzx <REG>,<REG>+<REG>*<NUM>+<NUM>
clang-9.0	jnb <NUM> movsxd <REG>,<REG> mov [<REG>*<NUM>+<NUM>],<REG> test [<REG>*<NUM>+<NUM>],<REG> mov [<REG>],<REG> movsxd <REG>,<REG>
clang-10.0	add [<REG>],<NUM> neg <REG> lea <REG>,<REG> nop [<REG>+<REG>*<NUM>+<NUM>] movzx <REG>,<REG>*<NUM>+<NUM> lea <REG>,<REG>+<REG>*<NUM>+<NUM>
clang-11.0	or <REG>,<NUM> add <REG>,<REG> lea <REG>,<REG>+<REG>*<NUM>+<NUM> mov <REG>,<REG>*<NUM>+<NUM> and <REG>,<REG> add [<REG>*<NUM>+<NUM>],<REG>
clang-12.0	mov [<REG>],<NUM> lea <REG>,<REG>+<REG>*<NUM>+<NUM> cmp [<REG>*<NUM>+<NUM>],<NUM> setz [<REG>*<NUM>+<NUM>] setb <REG> lea <REG>,<REG>
clang-13.0	cmp <REG>,<NUM> setnb <REG> adc <REG>,<NUM> test <REG>,<NUM> sub <REG>,<NUM> ja <NUM>
gcc-4.9.4	push <REG> sar <REG>,<NUM> sar <REG>,<NUM> cqp hlt
gcc-5.5.0	sar <REG>,<NUM> cqp push <REG> sar <REG>,<NUM> nop [<REG>*<NUM>+<NUM>] cmp <REG>,<REG>
gcc-6.5.0	jbe <NUM> inc <REG> jnb <NUM> inc <REG> jbe <NUM> mov [<REG>+<REG>*<NUM>+<NUM>],<REG>
gcc-7.3.0	hlt push <REG> hlt sub <REG>,<REG> push <REG>
gcc-8.2.0	xor <REG>,<REG> and [<REG>],<NUM> jnz <NUM> imul <REG>,<REG> sbb <REG>,<NUM> test <REG>,<REG>
gcc-9.4.0	mov [<REG>+<REG>*<NUM>+<NUM>],<NUM> lea <REG>,<REG>+<REG>*<NUM>+<NUM> cmp [<REG>],<REG> setnz [<REG>*<NUM>+<NUM>] and <REG>,<REG>*<NUM>+<NUM> test <REG>,<REG>
gcc-10.3.0	movups [<REG>*<NUM>+<NUM>],<REG> and <REG>,<NUM> shl <REG>,<NUM> bt <REG>,<REG> mov <REG>,<REG> scas
gcc-11.2.0	endbr64 jmp <NUM> endbr64 cmp <NUM>,<NUM> endbr64 test <REG>,<REG>

Table 7: Top-3 Important Features for Optimization Identification

Optimization Level	Top-3 Features
O0	sub <REG>,<NUM> mov [<REG>*<NUM>+<NUM>],<REG> jz <NUM> mov <REG>,<REG> push <REG> push <REG>
O1	mov [<REG>*<NUM>+<NUM>],<REG> add [<REG>],<NUM> nop [<REG>+<REG>*<NUM>+<NUM>] movsxd <REG>,<REG> and <REG>,<NUM> retn
O2	mov <REG>,<REG> push <REG> <ECALL> movsxd <REG>,<REG>*<NUM>+<NUM> and <REG>,<NUM> xor <REG>,<REG>
O3	nop [<REG>*<NUM>+<NUM>] cvtsi2ss <REG>,<REG> movaps <REG>,<REG> mov [<REG>],<REG> mov <REG>,<REG>
Os	mov <REG>,<REG> inc <REG> inc <REG> mov <REG>,<REG> mov [<REG>*<NUM>+<NUM>],<REG> jmp <NUM>
Ofast	ldmxcsr [<REG>*<NUM>+<NUM>] jnz <NUM> nop [<REG>+<REG>*<NUM>+<NUM>] nop [<REG>+<REG>*<NUM>+<NUM>] mov <REG>,<REG>*<NUM>+<NUM>